

Curación pendiente

Los 41 módulos que el grafo no cubre, con todo lo verificable ya resuelto.

Generado por `scripts/hoja_de_curacion.py` · se regenera, no se edita a mano

No queda nada pendiente

Los **10** módulos que la medición sigue marcando «sin nodo» están todos decididos, y **ninguno queda en T**. La decisión la tomó la medición; el *motivo* es lo único que viaja al futuro, porque es lo que va a decidir el próximo módulo parecido. Cada motivo tiene un disparador de revisión distinto, y T es la marca que no se revisa nunca: archivar bajo T algo que sí hay que revisar equivale a perderlo.

motivo	qué es	cuándo se revisa
alias	una presentacion del objeto de un nodo que YA existe; su nombre va en el campo 'lean' de ese nodo, no en uno nuevo	si cambia el nodo padre
huerfano	objeto legitimo cuya AREA no esta en el grafo: sin padre la arista no existiria y el nombre quedaria suelto	si entra el area
politica	TIENE objetos, pero es metalenguaje y su vocabulario es caro: sus tokens salen en media biblioteca	nunca, salvo decision explicita
cubierto	el nombre ya esta en el campo 'lean' de un nodo: el alias ya aplicado	si cambia ese nodo
T	ni objetos ni flechas	nunca

módulo	motivo	por qué
<code>AlgebraicTopology. SimplexCategory.Generators Relations.Basic</code>	alias	‘SimplexCategoryGenRel’ (línea 75) es SimplexCategory presentada por generadores y relaciones. AVISO: la equivalencia ‘SimplexCategoryGenRel $\simeq$ SimplexCategory’ NO esta probada en Mathlib —solo el funtor ‘toSimplexCategory’ (251) y las piezas de EpiMono y NormalForms—, así que «la misma categoría con otro nombre» es una conjetura, no un hecho verificado, y el alias apuntaría a través de un funtor del que aun no se sabe que sea equivalencia
<code>SetTheory.Ordinal.Notation</code>	alias	‘ONote’ (40) y ‘NONote’ (1129) son la forma normal de Cantor como dato computable, y ‘repr’ (69) es la flecha canónica de evaluación hacia ‘Ordinal’. NO es la identidad: es una presentación del objeto del nodo ‘ordinals’, que ya existe
<code>Topology.Category.TopCat. Basic</code>	cubierto	‘structure TopCat’ (32) ya es la identidad de ‘point-set-topology’. La convención del proyecto pone el envoltorio categorico en el campo ‘lean’ del concepto (group-theory lleva GrpCat), no en un nodo aparte
<code>Computability.AkraBazzi. SumTransform</code>	huerfano	‘structure AkraBazziRecurrence’ (60) empaqueta la recurrencia con sus hipótesis: hay objeto. La analogía con ‘prime-factorization’ NO se sostiene —aquel es un teorema sin estructura empaquetada—. Lo que falta es el padre: ‘algorithm-analysis’ esta marcado T y sin nombre
<code>Order.PFilter</code>	huerfano	‘PFilter’ (45) es una estructura con ‘instance : PartialOrder’ (77): objeto de pleno derecho. Lo que falta es el PADRE —no hay ni un concepto curado de teoría de ordenes, solo nodos generados y el área. Y el área ya está medio dentro sin nombre: ‘divisibility-gcd’, ‘subgroups-cosets’ e ‘ideals-quotient-rings’ son retículos y preórdenes colgando de otro sitio. AVISO PARA CUANDO ENTRE: su token es ‘filter’, de los más frecuentes de Mathlib; sin keyword declarada bajaría la precisión como hizo ‘different’
<code>Control.Bitraversable.Basic</code>	politica	‘class Bitraversable’ (48) es un objeto. Fuera por metalenguaje: es la interfaz de efectos de Lean
<code>Control.Fix</code>	politica	‘class Fix’ (35) es un objeto. Fuera porque su token sería ‘fix’, que sale en media biblioteca
<code>Control.Functor.Multivariate</code>	politica	‘class MvFunctor’ (32) es un objeto. Su token sería ‘functor’
<code>Control.Monad.Cont</code>	politica	‘class MonadCont’ (33) y ‘def ContT’ (48) son objetos. Su token sería ‘cont’
<code>Data.TypeVec</code>	politica	EL CASO QUE REFUTA LA MARCA DE FRENTE: define objetos (‘TypeVec’, 41), flechas con notación propia (‘Arrow’, 52, con ‘ $\implies$ ’), identidad (‘id’, 71) y composición. Fuera por metalenguaje y porque su token sería ‘type’. La puerta de las plazas ya lo para sin prohibirlo: no se le declara ninguna keyword, y esa es la razón

Si mañana la medición destapa módulos nuevos, esta hoja vuelve a llenarse sola.

1 · La marca. Es lo que sostiene el grafo, y no la decide nada automático:

C	una categoría	→	vértice
S	una subcategoría plena	→	vértice , y la inclusión es arista
F	un funtor o clase de flechas	→	arista <i>en este ambiente</i>
O	un objeto individual	→	vértice degenerado
T	ni objetos ni flechas	→	fuera

F **no** dice «esto no puede ser un vértice nunca». Eso es falso, y está demostrado que lo es en `FlechasComoObjetos.lean`: las flechas de C son exactamente los objetos de la categoría de flechas $\text{Arrow } C$, y los funtores son los objetos de la categoría de funtores $[C, D]$. Lo que dice es que **en este grafo** —cuyos objetos son conceptos y cuyas flechas son dependencias— la etiqueta nombra una flecha.

homology es F: aquí no es una colección que se pueda colimitar, es el funtor *a lo largo del cual se colimita*. prime-factorization es T: es un teorema, no un objeto.

2 · El padre. De qué concepto es especialización. La flecha va del general al específico.

3 · Cuáles nombres se quedan. Que exista no lo hace correcto: `Nat.Prime` es la noción de primo, `Nat.minFac` no lo es.

Lo que **no** hay que decidir, porque ya está verificado: si el nombre existe, en qué módulo vive, cuál es el canónico —las citas— y el DAG de `imports`. Los identificadores de abajo están **leídos de su declaración**, no deducidos de la ruta: de 447 deducidos así, 95 no existían.

Antes de dar por buena una tanda

```
python -m scripts.recuperacion_contra_proofnet
```

Precisión y cobertura contra 371 formalizaciones de oro, con su modelo nulo y sin gastar API.

Baseline hoy: 22,8 % / 18,0 % contra 1,45 % / 3,3 % — 15,7×. Si la precisión baja, esa tanda **no entra**.

Ya pasó: ofrecer los sustantivos de los nodos generados bajaba de 14,0 % a 11,5 %. Añadir vocabulario tiene coste.

Y la regla tiene letra pequeña. La primera tanda de 31 nodos bajaba la precisión a 19,9 % sin mover la cobertura, y la causa no era ningún veredicto equivocado —los 47 nombres los acepta `#check`— sino que el emparejador tokeniza el **id** y el **nombre** de cada nodo: `different-ideal` aportaba el token `different`, que sale en media biblioteca, y con eso gastaba una de las dos plazas del prompt. La puerta que lo arregla —*sólo se ocupa plaza con una keyword declarada*— subió la línea base de 21,3 % a 22,8 %. Si una tanda baja la precisión, mira primero si sus nodos entran en plaza por su nombre.